

```
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
import scipy.stats as stats
import matplotlib.pyplot as plt
```

Load and Clean the Data

```
df = pd.read_csv("/content/marketing_and_sales_data_evaluate_lr.csv.csv")

# 2. Check for missing values (NaNs)
print("Missing values per column before cleaning:")
print(df.isnull().sum())

# 3. Drop rows with missing values to clean our data
df = df.dropna()

# 4. Confirm the data is clean and look at the first 5 rows
print("\nCleaned Data Preview:")
print(df.head())
```

```
Missing values per column before cleaning:
TV          10
Radio        4
Social_Media  6
Sales        6
dtype: int64
```

```
Cleaned Data Preview:
   TV      Radio  Social_Media    Sales
0  16.0   6.566231    2.907983   54.732757
1  13.0   9.237765    2.409567   46.677897
2  41.0  15.886446    2.913410  150.177829
3  83.0  30.020028    6.922304  298.246340
4  15.0   8.437408    1.405998   56.594181
```

Exploratory Data Analysis (EDA)

```
print("Correlation with Sales:")
print(df.corr()['Sales'])

# Visualize the relationships using scatter plots
sns.pairplot(df, x_vars=['TV', 'Radio', 'Social_Media'], y_vars='Sales', height=5, aspect=0.8, kind='reg')
plt.show()
```

Correlation with Sales:

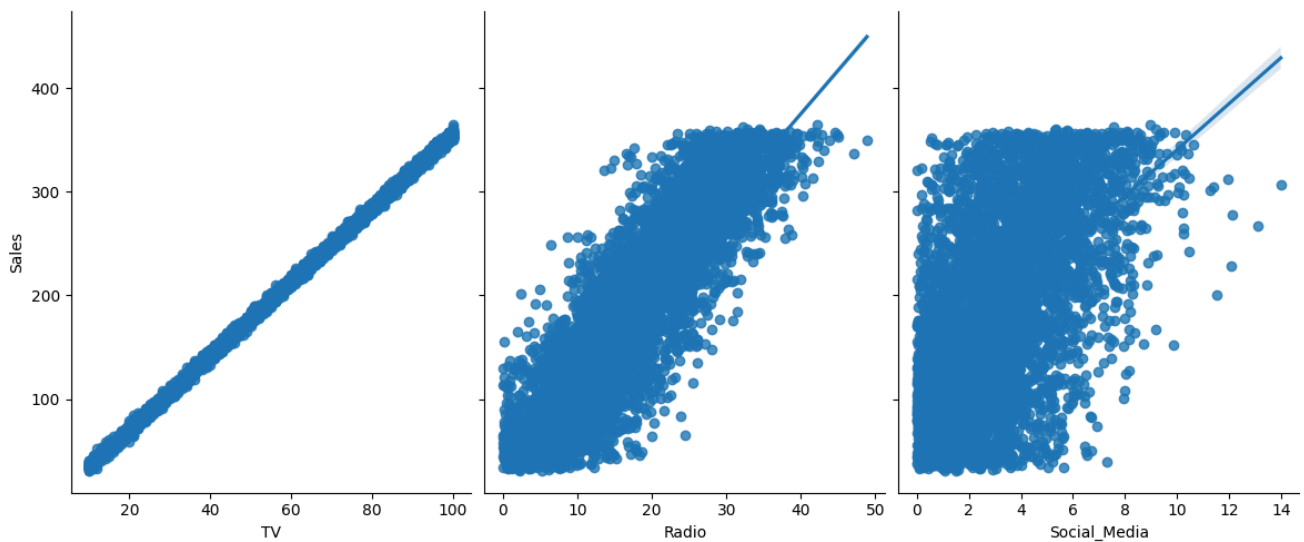
TV 0.999497

Radio 0.868638

Social_Media 0.527446

Sales 1.000000

Name: Sales, dtype: float64



Build the OLS Regression Model

```
# Define our dependent (Y) and independent (X) variables
```

```
Y = df['Sales']
```

```
X = df['TV']
```

```
# Add a constant (intercept) to the model
```

```
X = sm.add_constant(X)
```

```
# Fit the Ordinary Least Squares (OLS) model
```

```
model = sm.OLS(Y, X).fit()
```

```
# Print the comprehensive results summary
```

```
print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.999
Model:                  OLS      Adj. R-squared:            0.999
Method:                 Least Squares    F-statistic:          4.517e+06
Date:                   Mon, 15 Jun 2026    Prob (F-statistic):      0.00
Time:                   12:42:06    Log-Likelihood:         -11366.
No. Observations:       4546    AIC:                    2.274e+04
Df Residuals:           4544    BIC:                    2.275e+04
Df Model:                1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1325	0.101	-1.317	0.188	-0.330	0.065
TV	3.5615	0.002	2125.272	0.000	3.558	3.565

```

=====
Omnibus:                 0.052    Durbin-Watson:          1.998
Prob(Omnibus):           0.974    Jarque-Bera (JB):         0.031
Skew:                    -0.001    Prob(JB):                 0.985
Kurtosis:                 3.012    Cond. No.:                 138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Test Model Assumptions (Diagnostic Plots)

```
# Get the model residuals (errors)
residuals = model.resid

# Plot 1: Normality Check (Q-Q Plot)
# If the dots line up along the red diagonal line, our errors are normally distributed!
stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot (Assumption Check)")
plt.show()

# Plot 2: Homoscedasticity Check (Residuals vs Fitted)
# We want to see a random cloud of dots. No funnel patterns!
plt.scatter(model.fittedvalues, residuals)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Sales")
plt.ylabel("Residuals (Errors)")
plt.title("Residual Plot")
plt.show()
```

